

Reputation Engine - Technical Specification

Multi-tenant NFC tap-to-review platform with server-side routing, sentiment triage, reseller commissions, and Stripe subscriptions. Built to be assembled quickly with AI coding tools.

1. Core architectural principle

Write each card once. Control everything server-side. Every NFC card is encoded a single time with a permanent URL containing an unguessable id: `https://app.DOMAIN.com/t/{cardId}`. The card is never re-encoded. When tapped, the server resolves `cardId` → `business` → `current settings` and decides where to send the visitor. "Activating" or re-pointing a card is a database update, not physical access. This is what makes fulfillment instant and the system scalable.

Tap request flow

```
GET /t/:cardId
```

1. Look up card. If unassigned/disabled → generic fallback page.
2. Load business + brand + review_url + subscription flags.
3. Render branded rating page (server-rendered, fast, edge-friendly).
4. Visitor selects 1-5 stars (client POSTs to `/api/tap`).
 - 4-5: record tap, 302 redirect to `business.review_url`
 - 1-3: record tap, show private feedback form (public link ALSO shown = compliant)
5. On feedback submit → store + fire SMS (Twilio) + email (Resend) to owner.

Compliance requirement (non-negotiable in code): the low-rating branch must still surface the public review link. Do not hard-gate. Enforce "feedback-first, not review-blocking" at the component level so no reseller/business config can turn it into review gating (Google/Yelp/FTC violation).

2. Recommended stack

Chosen for speed of AI-assisted build, low run cost, and fast edge redirects. Swap equivalents freely; the architecture is stack-agnostic.

Framework	Next.js (App Router) - one codebase for marketing site, dashboards, tap pages, and API routes. (Alt: Lovable/React if building visually with AI.)
DB + Auth	Supabase (Postgres) - Row-Level Security for multi-tenant isolation, built-in auth, roles. (Alt: Postgres + Clerk/Auth.js.)
Hosting	Vercel or Cloudflare Pages/Workers . The <code>/t/:cardId</code> resolver should run at the edge for low-latency taps.
Payments	Stripe - subscriptions (\$15/mo), card-order checkout, reseller payouts via Stripe Connect (optional).
SMS	Twilio - instant low-rating alert to owner/manager.
Email	Resend - alerts, order confirmations, activation notices.
NFC write	NTAG213 chips + "NFC Tools" app (or bulk pre-encoding by supplier via CSV of URLs).

3. Roles & tenancy

Role	Can do
Super Admin (us)	Everything: manage resellers, businesses, card inventory/batches, fulfillment, billing, payouts, global analytics.
Reseller (rep)	Self-signup. Buy cards at wholesale, assign/activate cards to businesses, view own sales + commission ledger + payouts.
Business (client)	Set review destination URL, customize landing page (logo/colors/copy), read feedback inbox + star analytics, manage \$15/mo subscription.

Enforce isolation with Postgres RLS: a business row is readable only by its owner + assigned reseller + admin. Never rely on client-side checks alone.

4. Data model (minimum viable schema)

```

users      (id, email, role[admin|reseller|business], stripe_customer_id, created_at)
resellers  (id, user_id→users, display_name, commission_rate, payout_method, status)
businesses (id, name, logo_url, brand_color, review_url, review_platform,
            owner_phone, owner_email, reseller_id→resellers,
            subscription_status[none|active|past_due|canceled],
            stripe_subscription_id, created_at)
cards      (id // the unguessable cardId in the URL, e.g. 12+ char base62,
            business_id→businesses nullable until assigned,
            status[unassigned|active|disabled], batch_id, activated_at)
taps       (id, card_id→cards, rating[1-5], redirected_to, user_agent, created_at)
feedback   (id, business_id→businesses, rating, customer_name, customer_phone,
            message, resolved[bool], created_at)
commissions (id, reseller_id, business_id, stripe_invoice_id,
            type[card_sale|sub_month_1|sub_month_2], amount,
            status[pending|paid|reversed], created_at)
card_batches (id, quantity, printed[bool], notes, created_at)

```

cardId must be random and unguessable (e.g. 12-char nanoid), never sequential - otherwise the URL space can be walked to hijack or enumerate cards.

5. Key API endpoints

Route	Purpose
GET /t/:cardId	Public. Resolve card, render branded rating page. Edge-cached per card where possible.
POST /api/tap	Public. Record rating; return redirect target (4-5) or feedback-form flag (1-3).
POST /api/feedback	Public. Store private feedback; trigger Twilio SMS + Resend email to owner.
POST /api/cards/assign	Reseller/admin. Bind a card to a business and set status=active.
POST /api/checkout/cards	Reseller. Stripe Checkout for wholesale card orders.
POST /api/checkout/subscription	Business. Stripe Checkout for \$15/mo reputation subscription.
POST /api/webhooks/stripe	Stripe events. Drives activation + commission ledger (see §6).
GET /api/business/feedback	Business. Paginated feedback inbox + analytics.

6. Payments & commission logic (Stripe webhooks)

All money automation is webhook-driven so nothing is manual.

Stripe event	Action
<code>checkout.session.completed</code> (subscription)	Mark business <code>subscription_status=active</code> . If the card is unassigned/inactive, auto-activate it now (card goes live the moment payment clears).
<code>invoice.paid</code>	Count paid invoices on this subscription. 1st paid invoice → write <code>commissions(sub_month_1, \$15)</code> to the reseller. 2nd → <code>sub_month_2, \$15</code> . 3rd+ → no commission (100% company).
<code>charge.refunded</code> / <code>invoice.payment_failed</code>	If it matches a rep-credited invoice, set that commission <code>status=reversed</code> (clawback). Prevents gaming with signup-then-cancel.
<code>customer.subscription.deleted</code>	Set <code>subscription_status=canceled</code> . Card can revert to a generic review link or stay pointed (business decision).

Anchor commissions to successful paid invoices, not calendar months or signup date. This handles retries, failed cards, and re-subscribes correctly, and pairs with the refund clawback so reps are only paid on money actually kept.

7. NFC card production

- **Chip:** NTAG213 is sufficient (URL fits easily). ~\$2/card at low volume, less in bulk.
- **Encoding:** a single NDEF URI record = `https://app.DOMAIN.com/t/{cardId}` . Free "NFC Tools" app writes one in ~5 seconds; lock the tag to read-only after writing.
- **At volume:** pre-generate a batch of `cardId` s (status `unassigned`), export a CSV of URLs, and either (a) write them in-house or (b) hand the CSV to an NFC printing vendor for pre-encoding + custom print.
- Assignment to a business happens later in software, so generic stock can be produced ahead of demand.

8. Security checklist

- Unguessable `cardId` (nanoid 12+); rate-limit `/api/tap` and `/api/feedback` to stop spam/skew.
- Postgres RLS for every tenant table; server-side authorization on all dashboard mutations.
- Verify Stripe webhook signatures; make webhook handlers idempotent (dedupe on event id).

- Store only what you need from customers (name + phone are optional on feedback); post a privacy notice.
- Bot/abuse protection on public tap pages; cap taps per card per short window.
- PII (owner phone/email) encrypted at rest via provider defaults; least-privilege API keys per integration.

9. Build phases

Phase	Scope
PHASE 1 Prove the mechanic	Tap resolver, branded rating page, 4-5 vs 1-3 split, private feedback form, Twilio SMS + Resend email alert. One hardcoded demo business. Goal: a card you can tap live in front of an owner.
PHASE 2 Make it a product	Auth + roles, business dashboard (review URL, page customization, feedback inbox, analytics), card assignment/activation.
PHASE 3 The business engine	Public marketing site + reseller self-signup, Stripe card checkout + \$15/mo subscriptions, commission ledger + reseller dashboard, super-admin panel + fulfillment, batch/inventory tooling.

10. Integrations & env

```
# .env (server-only; never expose service keys to client)
NEXT_PUBLIC_APP_URL=https://app.DOMAIN.com
SUPABASE_URL= / SUPABASE_ANON_KEY= / SUPABASE_SERVICE_ROLE_KEY=
STRIPE_SECRET_KEY= / STRIPE_WEBHOOK_SECRET=
STRIPE_PRICE_SUBSCRIPTION=price_... # $15/mo recurring
TWILIO_ACCOUNT_SID= / TWILIO_AUTH_TOKEN= / TWILIO_FROM_NUMBER=
RESEND_API_KEY= / ALERT_FROM_EMAIL=
```

11. Extensibility (design for the endgame)

The platform is a lead-generation engine for higher-ticket website + AI services. Build the business dashboard with room for upsell modules and clean contact capture, so services can be cross-sold to accounts that already pay and trust the product. Keep `businesses` records rich (industry, contact, engagement) for later segmentation.

guidance.